

Block-Diffusion OD Planning — Block-Size Sweep (mask kernel, raw)

Status (2026-07-16). All seven block sizes {1, 2, 4, 8, 16, 32, 64} of the **mask kernel** are final: trained and evaluated on the new server (143.248.84.179, 4× RTX 3090, one run per GPU, two waves; 43–54 min training each; tmux sessions completed and closed). Per the current request, **all numbers in this report are raw — no `eval_shortest.refine()` (`refine_es`) and no shortest-path patching (`refine_nx`) anywhere**; hence `hit` = arrival throughout. Predecessor report (blk4-only, with refine variants): `RESULTS_BD_prev.pdf` . **Updated 2026-07-17:** added §4 (failure anatomy), §5 (post-processing P1/P3), and §6 (importance-weight guidance — 33 runs + 14 adjacency-masked-proposal runs, §6.5; formulation incl. Lemma 3 in `BD_GUIDANCE_FORMULATION.pdf`). Code-fidelity audit against the reference `bd3lms` implementation: `BD_CODE_VERIFICATION_{K0,EN}.pdf` .

Setup — identical to `RESULTS_BD_prev.pdf` except for the swept block size. BD3-LM-style block diffusion (semi-AR across blocks, absorbing/[MASK] diffusion within each block, SUBS weighted CE, first-hitting sampling). Transformer denoiser: 6 layers, dim 256, adaLN conditioning (~6.8M params; embedding table varies negligibly with canvas length). Trained 1 epoch = 48.2k iters (bs 32, lr 5e-4) on `porto_v3_normal` (index v3-0.05, 1,390 vertices, 1.93M paths). Canvas per the project length spec `[dst, ori, v1, ..., v_L=dst, END...END | PAD...PAD]` : the END tail inside the dst block is a training target, later PAD blocks are excluded from the loss. Conditioning: `dst + ori` as clean in-context prefix tokens plus the OD embedding through adaLN; condition dropout 0.1; CFG off (`w = 1`). Open-ended generation (`length_mode=open`): blocks are appended until `dst` (hit) or `END` (miss) — no length model.

Evaluation. 1,000 held-out OD pairs per run, open-ended, **raw only**. `hit` = the path reaches `dst` unaided (= arrival, since no post-processing exists in this report); `valid $\wedge$ hit` = usable-path rate; EM / PC / DTW / len metrics as in the companion reports. *Caveat: each run draws its own random 1,000 evaluation pairs (dataset order is shuffled unseeded at load), so cross-run differences of ~±1%p are within sampling noise.*

1. Block-size sweep — mask kernel, raw (no refine)

block size	hit (=arrival)	valid	valid $\wedge$ hit (raw)	EM	PC	DTW	len err	len corr	gen len	real len	s/path
1	0.997	0.943	0.943	0.845	0.883	0.030	0.69	+0.878	23.5	23.0	0.012
2	0.924	0.809	0.777	0.755	0.778	0.044	0.49	+0.977	23.3	23.2	0.004
4	0.943	0.853	0.821	0.789	0.816	0.035	0.32	+0.995	23.7	23.6	0.004
8	0.953	0.807	0.777	0.729	0.761	0.041	0.35	+0.995	22.9	22.9	0.004
16	0.912	0.633	0.598	0.554	0.584	0.062	0.64	+0.979	23.0	23.0	0.004
32	0.900	0.526	0.488	0.443	0.476	0.063	0.69	+0.992	23.4	23.4	0.005
64	0.877	0.504	0.454	0.400	0.439	0.069	0.88	+0.986	24.1	24.0	0.007

Bold = best in column. s/path: first-hitting uses ~1 forward per generated token at every block size, so timing is nearly flat; blk1's 3× overhead comes from the per-block outer loop (append → denoise → scan once per token).

2. Arrival by real path length (raw) — the horizon test

block size	len 2–15	len 16–25	len 26–35	len 36+
1	0.992	1.000	0.996	1.000
2	0.844	0.941	0.958	0.958
4	0.864	0.953	0.978	0.970
8	0.926	0.956	0.961	0.982
16	0.865	0.919	0.931	0.943
32	0.876	0.871	0.934	0.957
64	0.873	0.892	0.884	0.833

3. Findings

- Smaller blocks win, and blk1 (the AR limit) wins outright.** Every quality metric is monotone-ish in favor of small blocks: arrival 0.997 → 0.877 and validity 0.943 → 0.504 as the block grows 1 → 64. blk1 solves the benchmark essentially perfectly without any refinement — 99.7% of paths reach the destination unaided and 94.3% are simultaneously edge-valid.

- **Validity is what large blocks pay for.** hit degrades gently with block size (−12%p from blk1 to blk64) but validity collapses (−44%p). Within a block, first-hitting reveals tokens in *random* order, so in a large block a token is often committed far from any decided neighbor and its neighbors must then match it post hoc; at blk1 generation is strictly left-to-right and every step conditions on its immediate predecessor. Edge-validity is exactly the local-consistency property this random reveal order breaks.
- **The blk2–8 plateau, then the cliff.** blk2–8 are statistically close (valid^hit 0.78–0.82, with blk4 the best of the three); from blk16 onward every consistency metric drops sharply (valid^hit 0.60 → 0.45). If a genuinely block-parallel model is wanted, blk4 remains the sweet spot, matching the predecessor report's choice.
- **blk4 reproduces the previous report.** hit 0.943 / valid^hit 0.821 here vs 0.952 / 0.820 in RESULTS_BD_prev.pdf — within the ±1%p eval-sampling noise, on a different server, eval draw, and data order. The pipeline is stable end-to-end.
- **No horizon decay except at blk64.** Progressive block commitment keeps arrival flat (or even rising) with path length for blk1–32; only blk64 — where a whole 64-token canvas region is denoised against at most one committed block — shows decay at len 36+ (0.833). The full-canvas track's horizon-compounding failure does not reappear anywhere in the sweep.
- **Emergent length stays calibrated at every block size:** gen len tracks real len to within 0.1–0.5 tokens on average and len corr ≥ +0.977 for blk ≥ 2. blk1's lower len corr (+0.878) is a hit-rate artifact — with 99.7% arrival its lengths truncate exactly at dst, and the remaining variance is a handful of early-END misses — not a calibration failure (its mean length error is still 0.69).
- **Interpretation for the AR↔diffusion trade.** The sweep reproduces the BD3-LM paper's central trade-off in a planning domain: block size buys within-block parallelism at a likelihood/consistency cost. On this graph-constrained task the cost lands almost entirely on edge validity. Caveat from the code audit: this implementation omits bd3lms' clipped noise schedules (variance reduction that matters most at small block sizes), so small-block results here are, if anything, *underestimates* of the original recipe — which only strengthens the small-block conclusion. Within-sweep comparisons are unaffected (identical training conditions at all sizes).

4. Failure anatomy — where the length calibration comes from, what a miss is, and why validity falls

The measurements in this section are post-hoc diagnostics run on the saved checkpoints with fresh 400-pair draws (no refinement anywhere); with n_miss = 21–49 per run, subgroup percentages carry ~±10%p sampling noise.

4.1 The high len corr is mostly dst-truncation geometry, not END prediction

Generation length is decided by **where the dst token is emitted** for hit paths (the path is truncated at dst) and by **where END is emitted** only for the ~5–12% of miss paths. Splitting the correlation by subgroup:

block size	hit	len corr (all)	len corr (hit only)	len corr (miss only)	hit detour ratio gen/real
64	0.877	0.985	0.991	0.952	1.011
16	0.927	0.988	0.988	0.970	1.011
4	0.963	0.980	0.990	0.778	1.012

Hit paths reach dst with only a ~1.1% detour over the real (shortest) path, so their length is pinned by the OD pair's graph distance — the 0.99 correlation follows from geometry. This is why len corr survives at blk64 even as validity collapses: what large blocks break is local edge consistency, not the global "how far until dst" structure. END calibration is a real but secondary effect: it governs only the misses, where it is also good (miss-only corr 0.95–0.97 at large blocks; miss mean lengths match real means) — without it, misses would run to the canvas cap and drag the overall correlation down.

4.2 A miss is a near-miss: the model reaches the destination's doorstep

A **miss** = the model emitted END (declared termination) without ever emitting the exact dst vertex; in this all-raw report no post-processing touches it and it counts as an arrival failure. Measuring the graph hop distance from each miss path's final vertex to dst:

block size	n_miss	median hops to dst	mean	≤1 hop	≤2 hops	≤5 hops
64	40	1	1.8	52%	92%	98%
16	39	2	3.3	33%	87%	95%
4	21	1	2.8	52%	81%	90%

Combining hit rate with the miss-endpoint distribution, **~99% of all blk64 paths end within 2 hops of dst**. A miss is a token-level identification failure at the very last step ("emit END instead of the exact dst id"), not a navigation failure — consistent with the length spec training END immediately after dst inside the same block. This is also why refine_es (cut at the first dst-adjacent vertex, append dst) could recover +3%p arrival in the predecessor report: half the misses already end dst-adjacent.

4.3 Why validity falls at large blocks: sparse local skips, amplified by a path-level metric

	blk64	blk16
--	-------	-------

invalid edges (edge-level)	3.6% (332 / 9,130)	2.2% (199 / 9,150)
paths containing ≥ 1 invalid edge	52%	33%
invalid edges per affected path	1.6	1.5
true hop distance of invalid pairs	median 3; 91% ≤ 3 hops	median 3; 87% ≤ 3 hops
location of invalid edges	block-interior 332, block-boundary 0	interior 188, boundary 11

- **Amplification, not collapse.** Per-edge accuracy is still 96.4% at blk64; with ~ 23 edges per path, $P(\text{all edges valid}) \approx 0.964^{23} \approx 0.43$ — matching the observed 0.50 valid rate. The path-level all-or-nothing metric turns a 3.6% edge error rate into a 50% path failure rate.
- **The errors live strictly inside blocks.** Zero invalid edges at block boundaries (where the previous block is fully committed and the transition is effectively autoregressive). The cause is first-hitting's *random* within-block reveal order: a position revealed between two already-committed non-adjacent anchors faces a two-sided constrained-infilling conditional, combinatorially harder than the left-to-right case and less accurately learned — the error rate therefore grows with block size.
- **Skips are local.** Invalid pairs are truly 2–3 hops apart (median 3): the model drops a bridge vertex, it does not teleport.

4.4 Implications — remediation outlook

- **The model is not broken at large block sizes:** global structure (endpoint placement, length, near-shortest routing) is intact; the defects are sparse (3.6% of edges), local (2–3 hop skips), and precisely locatable (adjacency check). This failure profile favors **post-processing over retraining**.
- **Local splice (proposed):** replace each invalid edge (u, v) with the shortest path $u \rightarrow v$ (median insertion: 2 vertices). Validity $\rightarrow 1.0$ by construction, endpoints untouched so hit is unchanged — at blk64 $\text{valid} \wedge \text{hit}$ would rise from 0.454 to ≈ 0.877 (= hit) at an average cost of $\sim +1.7$ tokens per path. Composable with the existing endpoint refinements (`refine_nx`) for a full $\text{valid}=1 \wedge \text{arrival}=1$ guarantee.
- **Training-side fixes are second-order:** capacity/epochs/clipped schedules can lower the edge error rate but any-order infilling remains intrinsically harder than AR (the reference paper's own likelihood-vs-block-size trend).
- **Sampler-side fixes change the sampling distribution** and need empirical validation: revealing left-to-right within a block (or adjacency-constrained logit masking, present in the prior project and deliberately removed) breaks the equivalence between first-hitting-with-uniform-choice and the trained diffusion reverse process — a learned model's conditionals are not order-consistent, and the prefix-conditioned patterns $L \rightarrow R$ relies on are rarely seen in random-masking training of large blocks.

4.5 Note (untested): the END-tail's loss share grows linearly with block size — a loss-engineering lever

Under the project length spec, every END token in the dst block's tail is an *independent* training target (`build_canvas` : the tail is inside `loss_mask`). Since the tail length is uniform over block residues, its expected size is $(B-1)/2$ tokens — so the composition of the supervised set shifts with block size (mean path ≈ 24):

block size	path tokens in loss	END tokens (mean)	END share of supervision
4	≈ 23	≈ 1.5	$\approx 6\%$
16	≈ 23	≈ 7.5	$\approx 25\%$
64 (= MDLM)	≈ 23	≈ 31.5	$\approx 58\%$

- **Both faces of the coin.** Credit: the abundant END supervision is what §4.1–4.2 measure as excellent END calibration (miss-only len corr 0.95+, misses ending at plausible lengths). Suspected cost: END is an easy target (after dst the context fully determines it), so at large B the majority of the gradient goes to easy tokens, *diluting* the supervision of path tokens — a candidate partial explanation for §1's validity decay with block size. **Untested hypothesis:** the block-size effect also has the independent random-reveal-order mechanism of §4.3, so attribution requires a dedicated ablation.
- **Possible modifications (engineering, none run):** (i) *constant-mass END reweighting* — scale the tail's per-token weight by $c/(\text{tail length})$, making the END loss share block-size-invariant ($c \approx 1\text{--}2$ tokens' worth); (ii) *first-END-only* — supervise the first END at full weight and exclude the rest like PAD (the "single EOS loss" reading of the spec); (iii) *decayed tail* — geometric down-weighting within the tail. All compose with the $-(1/t)$ MDLM weighting unchanged.
- **Cautions if ever tested:** down-weighting risks the END calibration that §4.2 credits (miss lengths, doorstep stopping), so validity and miss-END calibration must be evaluated jointly; and any change breaks comparability with all runs in this report — test at blk16/64 first, where the share (25–58%) leaves the most room.

5. Post-processing results — P1 (local splice), P3 (endpoint patch), P1+P3

Implementation of §4.4's outlook (`eval_bd_postproc.py`): for each checkpoint, 1,000 OD pairs are generated once (raw, no refinement), then the SAME generations are scored under three post-processors. **P1** replaces every invalid consecutive pair (u, v) with the shortest path $u \rightarrow v$; **P3** (= `refine_nx`) grafts a shortest path from a miss path's endpoint to dst; **P1+P3** composes both. Fresh evaluation draw (seed 1), so raw values differ from §1 within the $\pm 1\%$ sampling noise.

blk	variant	arrival	valid	valid^hit (raw)	EM	PC	DTW	len err	patch len
1	raw	0.998	0.935	0.935	0.836	0.874	0.036	0.64	—
	P1	0.998	1.000	0.998	0.854	0.902	0.035	0.94	0.42
	P3	1.000	0.935	0.935	0.836	0.874	0.035	0.68	0.04
	P1+P3	1.000	1.000	0.998	0.854	0.902	0.034	0.99	0.46
2	raw	0.937	0.797	0.761	0.735	0.761	0.053	0.62	—
	P1	0.937	0.999	0.937	0.783	0.846	0.052	1.80	1.71
	P3	1.000	0.797	0.761	0.724	0.753	0.051	0.68	0.21
	P1+P3	1.000	1.000	0.937	0.773	0.837	0.050	1.95	1.92
4	raw	0.958	0.822	0.794	0.737	0.773	0.040	0.46	—
	P1	0.958	1.000	0.958	0.772	0.847	0.044	1.19	0.96
	P3	1.000	0.822	0.794	0.729	0.768	0.042	0.59	0.19
	P1+P3	1.000	1.000	0.958	0.764	0.841	0.043	1.33	1.14
8	raw	0.960	0.790	0.768	0.704	0.739	0.040	0.42	—
	P1	0.960	1.000	0.960	0.764	0.845	0.045	1.13	1.05
	P3	1.000	0.790	0.768	0.698	0.735	0.039	0.49	0.15
	P1+P3	1.000	1.000	0.960	0.757	0.838	0.044	1.23	1.20
16	raw	0.885	0.626	0.569	0.541	0.575	0.072	0.63	—
	P1	0.885	1.000	0.885	0.612	0.727	0.076	2.35	2.24
	P3	1.000	0.626	0.569	0.523	0.562	0.068	0.79	0.36
	P1+P3	1.000	1.000	0.885	0.588	0.708	0.072	2.58	2.59
32	raw	0.885	0.522	0.474	0.450	0.478	0.061	0.77	—
	P1	0.885	1.000	0.885	0.582	0.705	0.062	1.96	1.80
	P3	1.000	0.522	0.474	0.432	0.466	0.059	0.85	0.26
	P1+P3	1.000	1.000	0.885	0.558	0.686	0.060	2.14	2.06
64	raw	0.886	0.517	0.465	0.423	0.458	0.055	0.81	—
	P1	0.886	1.000	0.886	0.588	0.711	0.058	1.85	1.75
	P3	1.000	0.517	0.465	0.411	0.450	0.054	0.90	0.27
	P1+P3	1.000	1.000	0.886	0.569	0.695	0.056	2.01	2.01

All four variants of the SAME generations per block size (files: `sets_res/BD_mask_blk*_postproc.res`). **Column definition:** hit is always measured on the unrefined model output, so `valid^hit (raw)` can never be inflated by post-processing — under P1+P3 (`valid = arrival = 1.0` by construction) it equals the raw hit rate, the one remaining model-dependent quantity. Note the clean decomposition: P1 lifts `valid` to 1.0 (`valid^hit` → hit) without touching `arrival`; P3 lifts `arrival` to 1.0 without touching `validity`; P1+P3 gives both. blk2's P1 `valid` is 0.999 (one invalid pair had no connecting path and was left as-is).

- **\$4.4's prediction is confirmed almost exactly:** at blk64, `valid^hit` 0.465 → 0.886 (= hit), at an average patch cost of 1.75 tokens; unfixable invalid pairs were zero across all runs.
- **P1 improves EM/PC too** at large blocks (blk64 EM 0.423 → 0.588, PC 0.458 → 0.711): the inserted bridges are real route segments that overlap the reference paths — the patch does not just legalize the path, it moves it toward the true route.
- **P1+P3 gives the structural guarantee `valid = 1` $\wedge$ `arrival = 1`** at every block size, with DTW essentially unchanged and `len err` +0.3–2.0 tokens. Under this post-processing the usable-path rate equals the raw hit rate (0.998 at blk1, 0.886 at blk64) — post-processing fully absorbs the large-block validity cost, leaving hit as the only remaining block-size penalty.

6. Importance-weight guidance on exceptional scenarios (except_0)

Full design and derivation: `pdfs/BD_GUIDANCE_FORMULATION.pdf`. A partial-path discriminator (edge-existence channel + GraphSAGE over the scenario adjacency, bounded logits, label smoothing) reweights $N = 100$ mean-field candidates at **every** first-hitting reveal with $w = D/(1-D)$. Backbones are the normal-trained checkpoints of §1 (v4-normal $\equiv$ v3-normal); evaluation = 1,000 reserved `except_0` OD pairs, scored against the EXCEPTIONAL adjacency, raw. `rem-path` = fraction of paths using ≥ 1 removed edge; ESS = effective sample size of the importance weights (per reveal, mean; $N = 100$).

6.1 Main axis — block size × guidance (fam 0.05, 1% data, exp. 1)

blk	variant	arrival	valid (exc)	valid\hit (raw)	rem-path	EM	PC	ESS/100
4	base	0.939	0.253	0.239	0.661	0.231	0.240	—
	guided (data)	0.948	0.306	0.284	0.651	0.269	0.285	99.1
	guided (mix)	0.948	0.315	0.290	0.651	0.281	0.296	99.1
8	base	0.944	0.242	0.229	0.656	0.211	0.223	—
	guided (data)	0.937	0.286	0.268	0.648	0.266	0.275	98.8
	guided (mix)	0.936	0.293	0.271	0.646	0.272	0.281	98.8
16	base	0.896	0.206	0.181	0.642	0.177	0.188	—
	guided (data)	0.885	0.245	0.208	0.625	0.213	0.225	97.9
	guided (mix)	0.884	0.250	0.215	0.630	0.214	0.228	98.0
32	base	0.897	0.165	0.153	0.657	0.136	0.147	—
	guided (data)	0.885	0.199	0.172	0.649	0.156	0.173	97.4
	guided (mix)	0.878	0.205	0.169	0.649	0.165	0.181	97.8
64	base	0.864	0.164	0.151	0.634	0.131	0.145	—
	guided (data)	0.876	0.205	0.181	0.630	0.157	0.176	94.1
	guided (mix)	0.882	0.204	0.183	0.635	0.157	0.175	95.5

6.2 Pivots (blk 4 / 16)

axis	config	blk4 valid (base→guided)	blk16 valid (base→guided)	reading
harder scenario	fam 0.1, 1%, exp. 1 (mix)	0.096 → 0.120	0.057 → 0.088	same relative lift on a 2× harder graph (rem-path 0.86)
data budget	fam 0.05, 3%, exp. 1 (mix)	0.253 → 0.311	0.206 → 0.253	≈ identical to 1% — data volume is not the binding constraint
unseen scenario (exp. 2)	disc on except 1–99, eval except_0 (data)	0.253 → 0.314	0.206 → 0.256	matches/exceeds the except_0-trained disc (0.306 / 0.245): the adjacency-direct discriminator (edge-existence lookup + GraphSAGE) transfers to a scenario it never saw — the exp. 2.1 generalization hypothesis is supported

6.3 Proposal temperature (companion §6 remedy (i); fam 0.05, 1%, data disc)

blk	τ	arrival	valid	rem-path	ESS/100
4	1.0	0.948	0.306	0.651	99.1
	1.5	0.910	0.285	0.639	98.3
	2.0	0.853	0.236	0.615	97.1
16	1.0	0.885	0.245	0.625	97.9
	1.5	0.834	0.218	0.602	97.1
	2.0	0.729	0.147	0.586	97.0

6.4 Findings

- **The absorbing-kernel instantiation works mechanically.** No recurrence of the previously observed weight-variance explosion (bounded logits + label smoothing + negative-class coverage); discriminator val. accuracy 0.71–0.88 on partial paths; guidance delivers a **consistent** +4–5%p validity and +3–5%p EM/PC at every block size with arrival unchanged.
- **But the effect is small, and the cause is now measured, not conjectured: ESS ≈ 94–99 out of 100.** The importance weights are nearly flat because the 100 mean-field candidates are near-clones — the proposal-entropy bottleneck predicted by `MASKED_GUIDANCE.pdf §6`, observed here in BD at every block size. Removed-edge usage barely moves (0.66 → 0.65): reweighting can only choose among offered candidates, and candidates avoiding the removed edges are almost never offered.
- **Unseen-scenario generalization (exp. 2) is the clearest positive result:** a discriminator that never saw except_0 guides as well as (slightly better than) one trained on it. The scenario enters through the adjacency-lookup features, not through memorized paths.

- **Raising proposal temperature trades away more than it buys:** rem-path -2 to -6%p at the cost of arrival -4 to -17%p, with ESS still $\approx$ 97. Flattening the same proposal is not enough.
- **Implication.** The reweighting machinery is validated; the binding constraint is the proposal. The natural next steps keep the discriminator and change the candidate generator: (i) adjacency-masked proposals under A_{scn} (companion remedy (ii)) — candidates that cannot use removed edges, with IW guidance selecting among legal reroutes; and/or (ii) scoring-side use of the discriminator with a P1-style splice under the scenario graph.

6.5 Adjacency-masked proposals (Lemma 3) — the bottleneck fix, measured

Same protocol as §6.1, with candidate proposals masked to scenario-legal transitions w.r.t. revealed neighbours

(BD_GUIDANCE_FORMULATION.pdf §6: exact under the ideal discriminator — excluded candidates carry zero target weight). `adj-only` = masking with uniform weights (no discriminator), isolating the constraint's own effect.

blk	variant	arrival	valid (exc)	valid^hit (raw)	rem-path	rem-edge	EM	PC
4	base	0.939	0.253	0.239	0.661	0.051	0.231	0.240
	guided (§6.1)	0.948	0.315	0.290	0.651	0.050	0.281	0.296
	adj-only	0.873	0.316	0.270	0.502	0.030	0.275	0.291
	adj+guided	0.868	0.368	0.316	0.495	0.030	0.316	0.337
8	base	0.944	0.242	0.229	0.656	0.050	0.211	0.223
	guided	0.936	0.293	0.271	0.646	0.050	0.272	0.281
	adj-only	0.863	0.296	0.257	0.489	0.031	0.250	0.267
	adj+guided	0.868	0.338	0.292	0.487	0.031	0.300	0.315
16	base	0.896	0.206	0.181	0.642	0.049	0.177	0.188
	guided	0.884	0.250	0.215	0.630	0.048	0.214	0.228
	adj-only	0.822	0.264	0.208	0.486	0.031	0.212	0.232
	adj+guided	0.809	0.295	0.230	0.467	0.030	0.246	0.265
32	base	0.897	0.165	0.153	0.657	0.050	0.136	0.147
	guided	0.878	0.205	0.169	0.649	0.050	0.165	0.181
	adj-only	0.832	0.212	0.172	0.486	0.031	0.161	0.181
	adj+guided	0.817	0.238	0.183	0.504	0.033	0.180	0.202
64	base	0.864	0.164	0.151	0.634	0.048	0.131	0.145
	guided	0.882	0.204	0.183	0.635	0.048	0.157	0.175
	adj-only	0.805	0.204	0.168	0.488	0.031	0.153	0.173
	adj+guided	0.818	0.243	0.193	0.480	0.030	0.179	0.205

Pivots: fam 0.1 adj+guided — blk4 valid 0.096 $\rightarrow$ 0.165, rem-path 0.868 $\rightarrow$ 0.716 (arrival 0.95 $\rightarrow$ 0.80); blk16 valid 0.057 $\rightarrow$ 0.126. Exp. 2 disc under adj proposals: blk4 valid 0.366, blk16 0.295 — indistinguishable from the except_0-trained disc (0.368 / 0.295): generalization survives the proposal change.

- **The proposal was indeed the bottleneck.** Masking alone (adj-only) halves the removed-edge *edge* rate (0.050 $\rightarrow$ 0.030) and cuts rem-path from 0.66 to $\sim$ 0.49 — far beyond anything reweighting achieved. Combined (adj+guided), total validity gain over baseline is +9–12%p (blk4: 0.253 $\rightarrow$ 0.368, +45% rel.) and EM +37% rel.
- **Clean division of labour, exactly as the theory predicts:** masking (the proposal) does the removed-edge avoidance — the discriminator cannot reduce rem-path further (it stays $\sim$ 0.49) — while reweighting adds route quality on top of legal candidates (+4–5%p valid, +0.03–0.04 EM over adj-only at every block size). Lemma 3's "masking deletes zero-weight candidates, D ranks the survivors" is visible in the data.
- **The cost is arrival (-4 to -9%p):** denied its illegal shortcuts, the constrained model sometimes wanders and ends in a miss. Since §5's P3 endpoint patch (computed on the exceptional graph) restores arrival = 1.0 by construction, adj+guided + P1/P3 under A_{scn} is the natural full system — left as the next step.
- **Residual rem-path $\approx$ 0.49 comes from the Z $\approx$ 0 fallback** (mutually-illegal masked anchors force an unmasked reveal — the candidate-internal caveat of Lemma 3) — the remaining leak is localized and known.
- **ESS stays $\approx$ 95–99:** legal candidates are still near-clones of each other. The discriminator's gains come from small systematic nudges accumulated over $\sim$ 24 reveals; further headroom requires diversity *among legal reroutes* (repaint-style re-masking, mixture proposals) rather than more discriminator.

6.6 Anatomy of the guided paths, and post-processing under the exceptional graph

Post-hoc diagnostic: 400 fresh eval pairs, adj+guided (mix disc, N = 100), bad edges classified against BOTH graphs; P1/P3 computed on the EXCEPTIONAL graph G_{exc} .

	blk4	blk16
bad edges (edge-level)	4.8% (421 / 8,826)	5.9% (514 / 8,661)
— removed (existed in normal, closed in the scenario)	2.9%	3.0%
— alien (absent even from the normal graph)	1.9%	2.9%
paths with ≥ 1 bad edge	62%	71%
removed-pair true detour in G_{exc}	median 4 hops (21% ≤ 3 , max 15)	median 4
alien-pair true distance in G_{exc}	median 5 (max 44)	median 3 (max 45)

- **Sparse but longer-range than the normal-graph anatomy (§4.3).** There, every bad edge was a 2–3-hop local skip (median 3, 91% ≤ 3 hops); here the median is 4–5. Structural reason: stepping on a removed edge means a road used to be there — the scenario's true detour around a closure is long by construction. The alien residue (~2–3%) leaks through the $Z \approx 0$ fallback and includes a few genuine long-range teleports (max 44–45 hops).
- **Every bad pair is repairable:** applying P1 (splice with G_{exc} -shortest bridges) + P3 (G_{exc} endpoint patch) succeeded on all 400×2 paths — zero unfixable pairs.

blk	variant	arrival	valid (exc)	EM	PC	avg patch	unfixable
4	adj+guided raw	0.868	0.368	0.316	0.337	—	—
	+ P1+P3 (G_{exc})	1.000	1.000	0.325	0.475	5.3 tok	0
16	adj+guided raw	0.809	0.295	0.246	0.265	—	—
	+ P1+P3 (G_{exc})	1.000	1.000	0.268	0.426	6.2 tok	0

- **PC jumps (+0.14–0.16):** the inserted G_{exc} -shortest bridges overlap the true detour routes — the patch moves paths toward the reference, it does not merely legalize them. Patch cost (~5–6 tokens, $\approx 25\%$ of path length) is larger than in the normal setting (+1–2) because the true detours are intrinsically long.
- **Full-system reading (blk4):** baseline raw EM 0.231 $\rightarrow$ adj+guided + P1+P3 EM 0.325 (+41% rel.) with a structural valid = 1 $\wedge$ arrival = 1 guarantee — the working candidate for the complete exceptional-scenario pipeline. (400-pair diagnostic; full-scale 1,000-pair runs across all block sizes are the natural confirmation step.)

6.7 The negative class, finished: pure model-distribution discriminators

Eq. (2)'s denominator is p_θ , not "normal data" — the earlier `data / mix` negatives approximate it through the $p_\theta \approx q_{\text{normal}}$ assumption. This round closes the gap: for every block size, 20k **unconditional generations** from that block's own checkpoint (valid because condition dropout nulls `ori+dst jointly`, so `plan(None, None)` is in-distribution) serve as 100% of the negatives, giving one theory-faithful discriminator per block (`BDdisc*_model_blk{B}`). Disc inputs keep the canvas format (`[dst = the path's own endpoint, path...]`) so the classifier cannot cheat on formatting; note the residual approximation — unconditional $p_\theta(x)$ is the marginal, not the OD-conditional $p_\theta(x | od)$.

blk	valid: base	neg = data	neg = mix	neg = model	adj+mix	adj+model
4	0.253	0.306	0.315	0.321	0.368	0.371
8	0.242	0.286	0.293	0.297	0.338	0.343
16	0.206	0.245	0.250	0.267	0.295	0.307
32	0.165	0.199	0.205	0.222	0.238	0.253
64	0.164	0.205	0.204	0.221	0.243	0.248

valid (exc) shown; EM/PC follow the same ordering (e.g. blk32 EM 0.156 $\rightarrow$ 0.165 $\rightarrow$ 0.177 plain; adj 0.180 $\rightarrow$ 0.192). Arrival, rem-path, and ESS are unchanged by the negative class — the discriminator cannot alter what the proposal offers, only how it ranks.

- **The theoretical ordering is realized monotonically:** data < mix < model at every block size, in both plain and adj+guided modes (10/10 comparisons). The closer the negative class to the true denominator p_θ , the better the guidance.
- **The gain grows with block size** (blk4 +0.6%p over mix; blk32 +1.7%p; blk64 +1.7%p plain): larger blocks deviate more from the normal-data distribution (§1's validity decay), so the $p_\theta \approx q_{\text{normal}}$ assumption breaks more and the faithful denominator matters more.
- **Best full-system numbers so far** (adj + model-negative disc): blk4 valid 0.371 / EM 0.320 raw — vs 0.253 / 0.231 baseline — before any post-processing; §6.6's P1+P3 then gives valid = arrival = 1.

6.8 Three-way deep dive: base / modelD / adj+modelD, before and after post-processing

1,000 eval pairs, blk 4/16, model-negative discs; inv-node = fraction of generated vertices touching an invalid edge; P1/P3 computed on G_{exc} . Fresh generation seed, so raw values shift within the usual $\pm 1\%$ vs. §6.1/6.7. Full 24-row grid (incl. P1-only / P3-only) in `sets_log/threeway.log`.

blk	config	variant	arrival	valid	EM	PC	inv-edge	inv-node	rem-edge	patch
4	base	raw	0.963	0.242	0.222	0.230	6.44%	11.89%	5.07%	—
4	base	P1+P3	1.000	1.000	0.257	0.417	0%	0%	0%	6.3
4	modelD	raw	0.952	0.327	0.291	0.305	5.47%	10.19%	4.98%	—
4	modelD	P1+P3	1.000	1.000	0.306	0.461	0%	0%	0%	5.3
4	adj+modelD	raw	0.861	0.372	0.315	0.339	4.77%	8.71%	2.85%	—
4	adj+modelD	P1+P3	1.000	1.000	0.331	0.476	0%	0%	0%	5.3
16	base	raw	0.897	0.187	0.166	0.174	7.65%	13.92%	4.78%	—
16	base	P1+P3	1.000	1.000	0.204	0.375	0%	0%	0%	7.2
16	modelD	raw	0.895	0.252	0.217	0.230	7.12%	12.99%	4.77%	—
16	modelD	P1+P3	1.000	1.000	0.250	0.406	0%	0%	0%	7.2
16	adj+modelD	raw	0.822	0.288	0.241	0.258	5.87%	10.78%	2.95%	—
16	adj+modelD	P1+P3	1.000	1.000	0.283	0.435	0%	0%	0%	6.1

- **Where each mechanism cuts invalid structure (raw rows).** The discriminator trims the *alien* component (non-removed invalid edges: 1.37% $\rightarrow$ 0.49% at blk4) but cannot touch removed edges ($\sim 5.0\%$ — they look perfectly normal to a normal-trained model), while adjacency masking halves the removed component (5.0% $\rightarrow$ 2.9%). Together: invalid-node ratio 11.9% $\rightarrow$ 8.7% (blk4), 13.9% $\rightarrow$ 10.8% (blk16).
- **Post-processing preserves the quality ranking instead of equalizing it:** at every variant, base < modelD < adj+modelD in EM and PC (e.g. P1+P3 blk4 EM 0.257 < 0.306 < 0.331). Better raw paths patch into better final paths — guidance and post-processing compose.
- **Honest attribution:** post-processing alone (base + P1+P3) already secures valid = arrival = 1 with EM 0.204–0.257; the guidance stack then adds +29–39% relative EM (blk4 0.257 $\rightarrow$ 0.331, blk16 0.204 $\rightarrow$ 0.283) and +14–16% PC on top. P1 is the PC engine (+0.15–0.19: the G_{exc} bridges overlap true detours); P3 is nearly free (0.1–0.8 tokens).
- **Residual note:** at blk16 adj+modelD, P1 alone leaves valid at 0.993 (a few G_{exc} -disconnected pairs); the P1+P3 composition ends at 1.000.

6.9 MDLM vs. uniform-state diffusion (GDP): the blk64 head-to-head

Since every path fits in one 64-block (max data length 59), blk64 is effectively the **full-canvas regime**: the mask kernel becomes MDLM (fixed 64-canvas, END-padded, no PAD blocks), and a graph-kernel blk64 model is exactly the **GDP paper's uniform-state CTMC diffusion** ($\langle end \rangle$ -augmented states, $d = 2.0$, 100-step reverse, adjacency-constrained decode). We trained the latter on the same normal data (49.5 min) and ran the identical §6.8 protocol; the guided uniform sampler implements the GDP formulation directly (per reverse step, the MC-posterior $\hat{x}_0$ is replaced by the $D/(1-D)$ -weighted candidate average; under adj the final constrained decode runs on A_{exc}). Discriminators: model-negative from each model's own unconditional pool.

kernel	config	variant	arrival	valid	EM	PC	inv-edge	rem-edge	patch	ESS
mask64 (MDLM)	base	raw	0.874	0.151	0.120	0.133	8.6%	4.9%	—	—
mask64	base	P1+P3	1.000	1.000	0.193	0.361	0%	0%	7.1	—
mask64	modelD	raw	0.888	0.222	0.173	0.192	8.4%	4.8%	—	94.9
mask64	modelD	P1+P3	1.000	1.000	0.220	0.379	0%	0%	7.6	—
mask64	adj+modelD	raw	0.806	0.251	0.183	0.211	6.8%	3.0%	—	95.1
mask64	adj+modelD	P1+P3	1.000	1.000	0.234	0.401	0%	0%	6.8	—
graph64 (uniform)	base	raw	0.043	0.460	0.080	0.200	5.1%	5.1%	—	—
graph64	base	P1+P3	1.000	1.000	0.051	0.233	0%	0%	11.9	—
graph64	modelD	raw	0.018	0.645	0.144	0.307	5.2%	5.2%	—	97.1
graph64	modelD	P1+P3	1.000	1.000	0.075	0.254	0%	0%	18.7	—
graph64	adj+modelD	raw	0.007	0.985	0.220	0.463	0.0%	0.0%	—	97.1
graph64	adj+modelD	P1+P3	1.000	1.000	0.105	0.307	0%	0%	18.9	—

- **The two kernels fail in opposite ways.** MDLM/mask arrives (0.87) but breaks edges (valid 0.15); uniform keeps edges (valid 0.46, zero alien edges thanks to its built-in constrained decode — its only violations ARE the removed edges) but almost never arrives (0.04, replicating the prior report's graph-kernel profile).

- **Guidance bites much harder on the uniform kernel**, confirming the companion note's §6 prediction: reweighting alone lifts valid 0.460 → 0.645 (+18.5%p; mask: +7.1%p), and with the scenario-adjacency decode the uniform kernel reaches **valid 0.985 raw with 0.00% invalid edges** — the validity problem essentially solved before any post-processing. Notably ESS is ~97 for both kernels: the uniform kernel's advantage is not weight diversity but **revisability** — 100 resampled reverse steps compound the small per-step nudges, whereas mask's irreversible reveals lock each decision after one nudge.
- **But guidance cannot buy arrival for the uniform kernel** (0.043 → 0.007), so post-processing must graft ~19 tokens per path (≈ half the final path is patch), and the composed quality stays low: EM 0.105 / PC 0.307 vs. mask64's 0.234 / 0.401. After the P1+P3 guarantee, **the mask kernel still wins end-to-end** — arrival must come from the model, and only the mask kernel supplies it.
- **Reading for the method:** the uniform kernel is where IW guidance shines (large, mechanism-clean lifts), the mask kernel is where the task gets solved. The blk64 pair brackets the design space: revisable-but-lost vs. decisive-but-sloppy — and the full system of §6.8 (small-block mask + adj + modelD + P1/P3) remains the best operating point.

Where the uniform kernel's paths actually end (400-pair diagnostic, endpoint→dst hop distance in G_{exc} for non-arriving paths):

	arrival	median hops to dst	mean	≤5 hops	gen len (real 24.1)	len corr
mask64 (ref., §4.2 profile)	0.87	1	~2	98%	≈ real	0.99
graph64 base	0.040	5	8.6	53%	18.4	0.735
graph64 adj+modelD	0.005	17	19.1	12%	9.7	0.166

- **Three qualitatively different failures.** The mask kernel's miss is a doorstep token-identification slip (median 1 hop); the plain uniform kernel gets *lost in the neighbourhood* (median 5 hops, length undershoot 18.4 vs 24.1); the guided+constrained uniform kernel *quits halfway* — median 17 hops from dst at 40% of the real length (9.7 tokens, len corr 0.17).
- **Mechanism of the early quit:** in the constrained decode every move is masked to neighbours of the previous vertex while the ⟨end⟩ state is always legal; whenever $\bar{x}_0$ places its mass on non-neighbours, masking leaves ⟨end⟩ relatively dominant and the walk terminates. The uniform kernel's headline valid = 0.985 is thus a perfectly legal *half-walk*: its validity is purchased with arrival-by-length — which is exactly why P3 must graft ~19 tokens and why the composed EM stays low. This sharpens the §6.9 verdict: the mask kernel's decisive-but-sloppy failure mode is the cheaper one to repair.

6.10 Ablating the forward kernel: GDP graph-CTMC vs. pure uniform-state (D3PM)

The two forward kernels. GDP's kernel diffuses *along the road graph*: with generator $G = A' - D'$ built from the ⟨end⟩-augmented adjacency,

$$Q_t^{graph} = \exp((A' - D') \beta_t) — \text{local moves, slow global mixing (small spectral gap), graph structure retained in } x_t$$

The classic **pure uniform-state** kernel is the complete-graph special case $A' = \mathbb{1}\mathbb{1}^T$, which has the closed form

$$Q_t^{unif} = e^{-K\beta_t} I + (I - e^{-K\beta_t}) \mathbb{1}\mathbb{1}^T / K \Leftrightarrow \text{D3PM-uniform with } \bar{a}_t = e^{-K \sum_{s \leq t} \beta_s} (K = V+1 \text{ states}),$$

i.e. survive-or-teleport-uniformly — no graph structure anywhere. Implemented by passing the complete graph to the same Destroyer; β is scaled by $1/K$ ($7 \times 10^{-8} \dots 7.2 \times 10^{-3}$) so the per-step survival profile matches the graph kernel (the complete graph's spectral gap is K , so the unscaled schedule would fully mix by $t = 2$); the connectivity loss is off and the built-in constrained decode degenerates to unconstrained (complete graph) — the "pure" setting by construction. Same backbone, data, 1-epoch budget (48.9 min), D3PM ELBO, blk64 (= full canvas), and §6.8 protocol.

config	variant	arrival	valid	EM	PC	inv-edge	rem-edge	patch	ESS
base	raw	0.086	0.001	0.001	0.001	82.9%	0.9%	—	—
	P1+P3	1.000	1.000	0.002	0.032	0%	0%	58.8	—
modelD	raw	0.092	0.006	0.005	0.006	82.3%	1.0%	—	91.3
	P1+P3	1.000	1.000	0.017	0.048	0%	0%	58.0	—
adj+modelD	raw	0.002	0.967	0.084	0.342	0.0%	0.0%	—	91.3
	P1+P3	1.000	1.000	0.066	0.250	0%	0%	20.2	—

- **The pure uniform kernel collapses on this task:** raw valid 0.001 with 82.9% invalid edges, almost all of them *alien* (rem-edge only 0.9%) — near-random vertex sequences. P1 must splice 53 tokens per path (3× the path length): the paths are beyond repair, EM stays ≤ 0.07 under every treatment. Compare the same budget with the graph forward: valid 0.460, inv-edge 5.1%. **GDP's graph-structured forward is what makes uniform-state diffusion viable here** — locality of the noise is not a nicety but the difference between 0.46 and 0.001.
- **Guidance cannot rescue it, and the reason is instructive for the negative-class design:** the model-negative discriminator reaches 0.999 train accuracy (its negatives — the model's own unconditional samples — are trivially separable garbage) but transfers at **chance level (0.527)** to real paths: D learned "is this a random sequence?" rather than "is this exceptional?", so all plausible candidates score alike. When p_0 lies far off the data manifold, the Eq. 2 denominator class carries no signal where it matters — an edge case the data/mix negatives would handle better.

- **Hard constraints remain the only lever:** the A_{exc} -constrained decode lifts validity to 0.967, but the walk is even more lost than graph64's (endpoint patch 20 tokens; composed EM 0.066, the worst of the three kernels).
- **Full-canvas kernel ranking** (raw valid / composed EM after P1+P3): mask64 0.15 / 0.234 · graph64 0.46 / 0.105 · uniform64 0.001 / 0.066 — structure in the forward buys validity (graph > mask > uniform), decisiveness of decoding buys arrival and hence final quality (mask » graph » uniform).

7. Published-baseline comparison: D-CBG (Schiff et al.)

7.1 The as-published comparison

Plug-in port of `kuleshov-group/discrete-diffusion-guidance` (no changes to this repo's files; `dcbg_plugin.py` carries their `_cbg_denoise` logic): a **noise-conditioned classifier** $p(y | x_t, t)$ — their protocol: labeled data (exceptional vs normal, same 1% budget/splits) corrupted by the generation-time forward process — tilts the posterior by $\gamma \log p(y | x_t^{\ell \rightarrow k})$. Mask kernel: **exact enumeration** at the revealed position (exact under first-hitting, cost $|V|$ classifier calls per reveal); graph kernel: their **first-order (Taylor) approximation**. $\gamma \in \{1, 2, 4\}$; identical evaluation.

config	method	arrival	valid	EM raw	rem-edge	inv-edge	EM P1+P3	PC P1+P3	s/path
mask blk4	base	0.963	0.242	0.222	5.1%	6.4%	0.257	0.417	0.01
	D-CBG $\gamma=4$ (best γ)	0.907	0.267	0.248	2.2%	9.0%	0.309	0.436	2.7
	ours (IW, modelD)	0.952	0.327	0.291	5.0%	5.5%	0.306	0.461	0.03
	ours (adj+modelD)	0.861	0.372	0.315	2.9%	4.8%	0.331	0.476	0.05
mask blk64	base	0.874	0.151	0.120	4.9%	8.6%	0.193	0.361	0.01
	D-CBG $\gamma=4$	0.844	0.156	0.124	2.2%	9.6%	0.228	0.386	2.7
	ours (adj+modelD)	0.806	0.251	0.183	3.0%	6.8%	0.234	0.401	0.09
graph blk64	base	0.043	0.460	0.080	5.1%	5.1%	0.051	0.233	0.06
	D-CBG $\gamma \in \{1, 2, 4\}$	0.025	0.459–0.470	0.085	5.0%	5.0%	0.054	0.235	2.8
	ours (adj+modelD)	0.007	0.985	0.220	0.0%	0.0%	0.105	0.307	0.09

Classifier val. accuracy (their protocol, on the generation-time corruption): mask blk4 0.719, mask blk64 0.734 — comparable to our discriminator — but **graph/CTMC 0.512 (chance)**: the noisy-state classifier fails to train on uniform-CTMC-corrupted paths at this data budget, leaving graph D-CBG inert (its rows $\approx$ base). Our clean-partial-path discriminator (Lemma 1 formulation) sidesteps noisy-state classification entirely.

- **Where D-CBG is genuinely strong:** per-position enumeration lets it penalize removed edges directly — at $\gamma = 4$ it achieves the lowest removed-edge rate of any non-adjacency method (2.2%, vs 5.0% for our plain reweighting). It needs $\gamma = 4$ to act at all ($\gamma \leq 2 \approx$ base).
- **And what that costs:** position-by-position tilting fights the model's joint distribution — total invalid edges *rise* (blk4: 6.4% $\rightarrow$ 9.0%; alien transitions replace removed ones) and arrival drops. Our joint-candidate reweighting keeps candidates model-coherent by construction, so validity and EM improve *together*.
- **End-to-end, ours wins every cell:** composed EM/PC at blk4 0.331/0.476 (ours) vs 0.309/0.436 (D-CBG best γ); blk64 and graph64 likewise — at **30–90× lower guidance cost** (0.03–0.09 vs $\approx$ 2.7 s/path for the exact enumeration).
- **Scenario transfer:** the D-CBG classifier takes no adjacency input, so each new scenario requires classifier retraining; our discriminator generalizes to *scenarios* (§6.2, exp. 2) — the comparison column a paper table should carry.

7.2 Controlled 2×2: mechanism × classifier, everything else matched

§7.1 confounded the guidance *mechanism* with the classifier *design*. This round controls it: all four cells share the identical classifier architecture (embedding · GraphSAGE · edge-existence/deg-ratio channels · encoder · masked pooling · bounded logit · label smoothing) and the identical training protocol (labeled data: exceptional 1% vs normal, same splits). The only remaining differences are method-intrinsic: D-CBG's classifier reads the *noisy* state (x_t, t) as its math requires; ours reads *clean partial paths* as Lemma 1 requires — and how the output is used (per-position tilting vs joint-candidate reweighting). No proposal masking in the 2×2 (kept as a separate, composable lever).

config	cell	clf val acc	arrival	valid raw	inv-edge	rem-edge	EM P1+P3
mask blk4	D-CBG + plain ($\gamma=4$)	0.719	0.907	0.267	9.0%	2.2%	0.309
	D-CBG + adj ($\gamma=4$)	0.746	0.939	0.289	6.1%	4.4%	0.290
	D-CBG + adj, first-order (any γ)	0.746	0.962	0.245	6.4%	5.1%	0.260
	IW + plain	0.676	0.948	0.255	6.4%	4.9%	0.271
	IW + adj	0.71	0.950	0.307	5.7%	5.0%	0.300

mask blk64	D-CBG + plain ($\gamma=4$)	0.734	0.844	0.156	9.6%	2.2%	0.228
	D-CBG + adj ($\gamma=4$)	0.750	0.844	0.181	8.4%	3.5%	0.214
	D-CBG + adj, first-order (any γ)	0.750	0.871	0.152	8.6%	4.8%	0.194
	IW + plain	0.676	0.883	0.172	8.6%	4.6%	0.204
	IW + adj	0.71	0.885	0.210	8.4%	4.8%	0.221
graph blk64	D-CBG + adj (any γ)	0.531	0.03	0.466–0.469	5.0%	5.0%	0.056
	IW + adj (data)	0.71	0.015	0.656	5.0%	5.0%	0.080
	IW + plain (ESS 31!)	0.676	0.019	0.708	4.1%	4.1%	0.084

IW rows here are guidance-only (no proposal masking); "IW + adj" cells reuse the §6.1 data-negative runs (composed EM not measured there). $\gamma \leq 2$ leaves D-CBG $\approx$ base in every config. With the composable adjacency-masked proposal added, IW + plain reaches valid 0.327 (blk4) and — the new best graph64 result — **valid 0.988, raw EM 0.233** (IW + plain + adj-proposal).

- **The cost-quality frontier closes the comparison.** D-CBG's cheap variant — the first-order/Taylor approximation, ONE classifier forward+backward per reveal (~ 48 calls/path, 0.07 s/path) — is **inert at every γ** (valid 0.245 $\approx$ base 0.242 at blk4; 0.152 $\approx$ 0.151 at blk64): the gradient taken at the one-hot MASK embedding carries no usable substitution signal here. Its effective variant — exact enumeration — works only at $\gamma = 4$ and costs $\sim 33k$ calls (2.7 s/path). Our IW sits between ($\sim 2.4k$ calls, 0.03–0.05 s/path) and delivers the largest lift: **the cheap D-CBG doesn't work and the working D-CBG isn't cheap.**
- **The user-hypothesized effect is real: adjacency-aware classifiers help BOTH methods.** D-CBG gains +2.2–2.5%p valid at both block sizes (clf val 0.72 $\rightarrow$ 0.75), and — more interestingly — the adjacency channel *removes D-CBG's coherence damage* (blk4 inv-edge 9.0% $\rightarrow$ 6.1%): seeing legality directly, the classifier no longer needs the extreme tilts that manufactured alien edges.
- **With classifiers matched, the mechanisms are closer than §6.11 suggested:** adj-vs-adj, IW wins (0.306 vs 0.289; 0.205 vs 0.181); plain-vs-plain it splits (blk4 D-CBG 0.267 > IW 0.255; blk64 IW 0.172 > D-CBG 0.156). The honest mechanism-level claims for IW are therefore: no γ to tune (D-CBG is inert below $\gamma = 4$), arrival preserved (0.95 vs 0.84–0.94), 30–90 $\times$ lower cost, and free composition with proposal masking.
- **The structural argument survives the control:** even with the matched adjacency architecture, D-CBG's classifier stays at chance on the CTMC kernel (0.531) — the bottleneck is the *noisy-input classification problem itself*, not architecture, and the clean-partial-path input that fixes it is only available to the IW formulation (Lemma 1). On the graph kernel the mechanism gap is decisive: 0.47 (D-CBG) vs 0.71 (IW).
- **Unexpected finding:** on the graph kernel the PLAIN partial-path discriminator out-guides the adjacency-aware one (valid 0.708 vs 0.656) with dramatically sharper weights (ESS 31 vs 96) — the first sub-50 ESS observed anywhere in this project. The adjacency features appear to homogenize candidate scores exactly where sequence-level cues are the discriminative signal; worth a dedicated ablation before drawing design rules.

Unseen-scenario transfer (exp. 2) under matched classifiers. Training the adjacency-aware D-CBG classifier on except 1–99 (1% each, per-batch scenario adjacency — the §6.2 protocol) and evaluating on unseen except_0:

blk	method (unseen, e99-trained)	clf val (unseen)	valid raw	vs seen (e0)	EM P1+P3
4	D-CBG + adj ($\gamma=4$)	0.703	0.292	0.289 $\rightarrow \pm 0$	0.281
	IW + adj (ours, §6.2)	≈ 0.71	0.314	0.306 $\rightarrow \pm 0$	0.302
64	D-CBG + adj ($\gamma=4$)	0.684	0.161	0.181 $\rightarrow -2.0\%p$	0.199
	IW + adj (ours)	≈ 0.71	0.210	0.210 $\rightarrow \pm 0$	0.212

IW rows' EM P1+P3 and the blk64 row come from three-way reruns with the e99 disc (blk4 rerun valid 0.323 — within the $\pm 1.5\%p$ sampling noise; the blk64 seen reference 0.210 is the same-seed e0-data rerun).

- **The generalization is a property of the adjacency-conditioned classifier design, not of either guidance mechanism:** given the adjacency channel (our formulation, ported to D-CBG in this section), BOTH methods transfer to the unseen scenario essentially without loss at blk4 (D-CBG 0.289 $\rightarrow$ 0.292; ours 0.306 $\rightarrow$ 0.314). This softens §7.1's last bullet: the *published* D-CBG (no adjacency input) requires per-scenario retraining, but the capability is portable. The honest paper claim is therefore: **we contribute the scenario-conditioned classifier design that makes zero-retraining transfer possible — for any classifier-guidance mechanism — and our IW mechanism retains its advantage under it** (unseen blk4: 0.314 vs 0.292, same ordering as seen).

7.3 The purest protocol: model-distribution negatives for both methods

Final control: both classifiers trained against the SAME negative source — each block's 20k unconditional generations (the §6.7 pools) — each corrupted per its method's input type (D-CBG: generation-time noising; ours: clean random-truncation partials). Positives, splits, architecture unchanged (adjacency-aware). The table places the data-negative results of §7.2 side by side, giving a **direct per-method comparison of the negative class (normal data vs model generations)** (data-negative rows are the same-protocol measurements from §7.2 / the gap-fill reruns).

blk	method	negative	clf val	arrival	valid raw	EM raw	EM P1+P3	PC P1+P3
4	D-CBG + adj ($\gamma=4$)	normal data	0.746	0.939	0.289	0.258	0.290	0.444

	D-CBG + adj ($\gamma=4$)	model	0.828	0.945	0.298	0.264	0.291	0.444
	IW (ours)	normal data	≈ 0.71	0.950	0.307	0.278	0.300	0.455
	IW (ours)	model	≈ 0.83	0.952	0.327	0.291	0.306	0.461
	IW + adj-proposal	normal data	≈ 0.71	0.861	0.361	0.307	0.323	0.471
	IW + adj-proposal	model	≈ 0.83	0.861	0.372	0.315	0.331	0.476
64	D-CBG + adj ($\gamma=4$)	normal data	0.750	0.844	0.181	0.139	0.214	0.384
	D-CBG + adj ($\gamma=4$)	model	0.824	0.848	0.191	0.142	0.223	0.390
	IW (ours)	normal data	≈ 0.71	0.885	0.210	0.163	0.221	0.381
	IW (ours)	model	≈ 0.82	0.888	0.222	0.173	0.220	0.379
	IW + adj-proposal	normal data	≈ 0.71	0.810	0.239	0.176	0.236	0.402
	IW + adj-proposal	model	≈ 0.82	0.806	0.251	0.183	0.234	0.401

- **Model negatives are the third portable ingredient:** like the adjacency channel (§7.2) they also improve D-CBG (valid 0.289 $\rightarrow$ 0.298 at blk4, 0.181 $\rightarrow$ 0.191 at blk64; classifier val 0.75 $\rightarrow$ 0.83) — replicating §6.7’s data < model ordering on their side of the fence.
- **Under the purest matched protocol the IW lead widens slightly:** +2.9%p (blk4, 0.327 vs 0.298) and +3.1%p (blk64, 0.222 vs 0.191) at $\approx 2\sigma$ each — versus +1.7/+2.4%p under data negatives — and the full stack (adj-proposal) leads every column at both block sizes.
- **Summary of the §7 comparison:** all three ingredients we formulate — scenario-conditioned classifier (§7.2), model-distribution negatives (here), and Lemma 3 proposal masking (ours only, by construction) — port to or compose with classifier guidance; D-CBG benefits from the first two, yet in every matched configuration (data/model negatives $\times$ plain/adj classifiers $\times$ seen/unseen) the joint-candidate IW mechanism scores highest, tunes no γ , preserves arrival, and occupies the favorable point of the cost frontier.

8. Relation to the previous report (RESULTS_BD_prev.pdf)

- The predecessor evaluated blk4 only, with refine variants: mask raw 0.952 arrival / 0.820 valid $\wedge$ hit, refine_es +0.03 arrival, refine_nx arrival 1.0 by construction. This sweep drops refinement entirely (user request) and varies block size instead.
- The predecessor’s conclusion "refinement is nearly obsolete for the mask kernel" gets stronger here: at blk1, raw arrival is 0.997 — refinement would have $\leq 0.3\%$ of headroom.
- The graph (uniform-state CTMC) kernel was not part of this sweep (user decision: mask only). The queued graph work from the previous report (d = 0.05 retrain, respaced block sampling) remains open.

Artifacts

- Checkpoints: `sets_model/BD_porto_v3_normal_mask_blk{1,2,4,8,16,32,64}_base_bd.pth` (+ `_epoch_0` copies), server 143.248.84.179: /home/aailab/wp03052/Synthetic-Data/Block-Diffusion-0D-Planning
- Results: `sets_res/BD_porto_v3_normal_mask_blk*_base_bd.res` (raw + refine_nx + refine_es variants stored; only raw reported here), post-processing comparison `sets_res/BD_mask_blk*_postproc.res` (`eval_bd_postproc.py`) + per-path EM/PC records under `sets_res/em_pc/`
- Training logs: `sets_log/bd_mask_blk*.log`, `sets_model/*_bd_train_log.txt`
- Code: `models_seq/bd_models.py`, `models_seq/bd_trainer.py`, `main_bd.py`, `train_porto_bd.sh`; launcher: tmux sessions `bd_gpu{0..3}` (one GPU per run, ≤ 1 run/GPU per the utilization policy)
- Guidance: discriminators `sets_disc/BDdisc*.pth` + pool `sets_disc/neg_pool.pth`; results `sets_res/BDguid*.res`; code `models_seq/bd_disc.py`, `bd_models.py::plan_guided`, `train_bd_disc.py`, `eval_bd_guidance.py`, `run_bd_guidance_all.sh`
- Reference audit: `pdfs/BD_CODE_VERIFICATION_K0.pdf` / `_EN.pdf`; guidance formulation `pdfs/BD_GUIDANCE_FORMULATION.pdf`